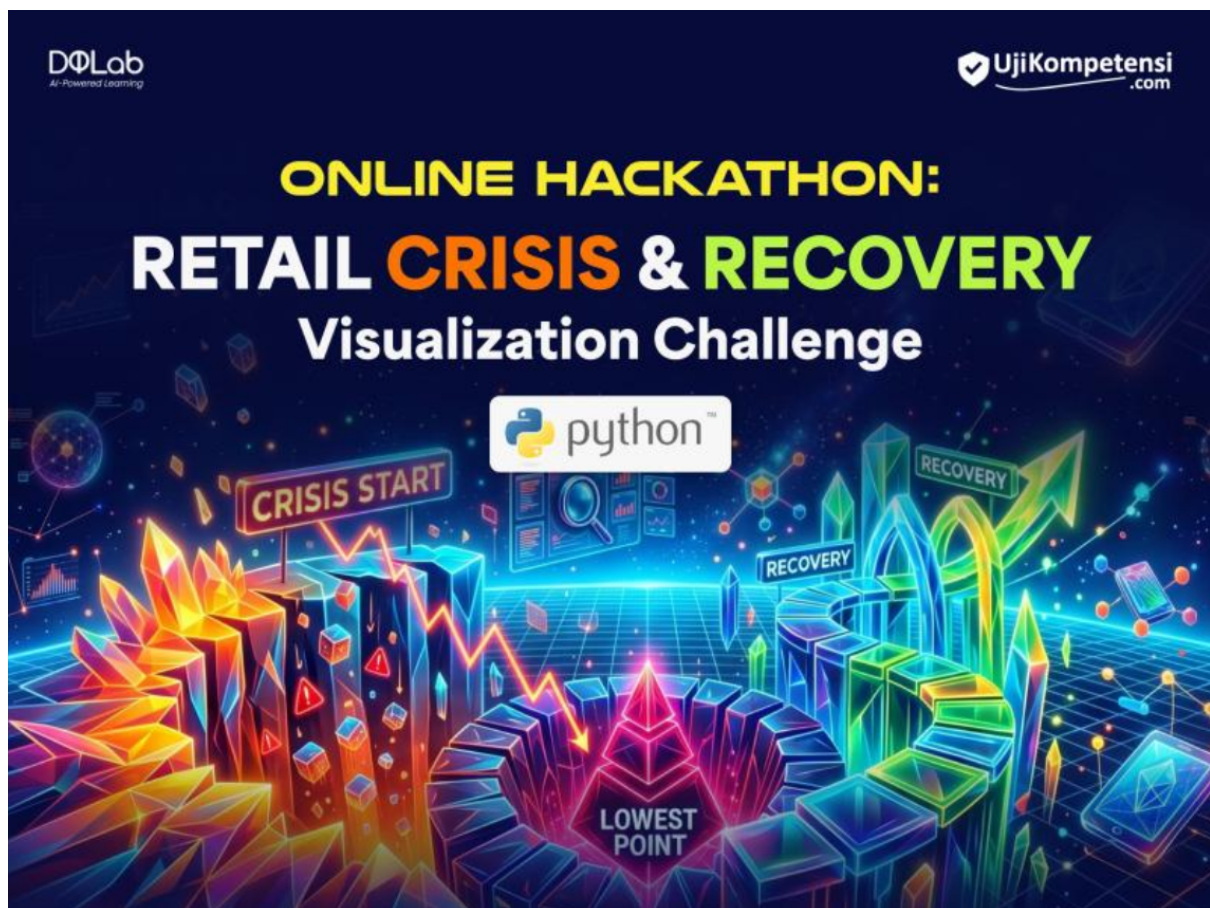


Table of Contents

- Python Hackathon DQLab x UjiKompetensi 2
- What Needs to Be Built and Submitted? 3
- Provided Dataset 3
- Expected Output from 'solusi-retail.py' Script 4
 - Excel File 4
 - Conditions for Generating Rising Star 4
 - Conditions for Generating Potential Packaging ... 5
 - Rising Star Visualization Specifications (Index and Actual) .. 5
- Python Version and Allowed Libraries 7



Python Hackathon DQLab x UjiKompetensi

DQFresh Mart Retail is a retail store (mini mart) with only one branch. For years, the company has been very successful in sales with traditional flagship products.

However, over the last 6 months, management began facing a serious problem: total sales value kept declining. Visibly, foot traffic also appeared to drop. The store manager initially assumed this condition was simply due to economic weakening. Therefore, the store's initial strategy was to:

- maintain bestseller products
- reduce experimentation with new products
- increase stock of the best historical products
- reduce inventory risk

However, after some time, Sophia, the store manager, began to feel something didn't add up. Sophia started her own investigation by analyzing the store's internal data:

- Sales transaction data
- Daily stock data

While conducting a deeper analysis, she found a pattern not visible in the store's main dashboard. Several SKUs that were not visible turned out to show consistent sales growth. However, because their total revenue contribution was still small, the system continued to regard these products as ones that didn't need attention. These SKUs also went unnoticed by cashiers during interviews, because they often went out of stock.

From this discovery, Sophia finally decided to fix the situation by increasing stock for these products and creating product bundles with other products that historically were frequently purchased together.

Participants are expected to technically produce the same analysis as Sophia, using a Python script, with the details (excluding daily stock data) explained in the following section.

What Needs to Be Built and Submitted?

Participants need to submit one Python script named `solusi-retail.py` which, when run with the command `"python solusi-retail.py"`, will produce three files in the working folder where the script is run:

- `retail_insight.xlsx`
- `rising_star_index.png`
- `rising_star_actual.png`

The script must be submitted via email with the subject name: "Solusi hackathon untuk soal 'HACK-2026-PYTHON-01'"

Only one script may be sent as an attachment with that email. No other files, including the three output result files, may be included in the email.

Provided Dataset

Participants will receive several dataset files whose metadata is the same as what Sophia processed, namely:

1. Sales Transaction Data Filename: `sales_transaction.csv` For this hackathon, nothing needs to be cleansed from this dataset, and for simplification, the data period is only 30 days.
 - o `nomor_struk`: receipt or invoice number of the transaction
 - o `tgl_transaksi`: the date the transaction was made
 - o `kode_produk`: code of the product sold
 - o `nama_produk`: name of the product sold
 - o `jumlah_terjual`: quantity of product sold
 - o `harga`: unit price of the product
 - o `total_nilai`: total sales value (price multiplied by quantity sold)

Expected Output from 'solusi-retail.py' Script

Excel File

An Excel file named `retail_insight.xlsx` containing one tab with the following names:

- **Rising Star**: products that are "invisible" because their moving average keeps increasing, but if aggregated using traditional methods like top 10 products, these products would not appear. (Instructions for obtaining these products are below.)
- **Potential Packaging**: combinations of products in the form of frequent itemsets generated by the Apriori algorithm.

The layout and example of the first row of results will be provided together with this problem in an Excel file named `retail_insight_example.xlsx`.

Conditions for Generating Rising Star

Use the pandas and matplotlib libraries to process the data according to the following technical requirements:

A. Data Smoothing:

- Calculate the Moving Average (MA) of total sales value with a 3-day time window for each product, to minimize extreme daily fluctuations.

B. Identifying Rising Trends:

- A product is declared to be in a "Rising Trend Session" if today's MA value is higher than the previous day's MA value.
- Count how many days this increase occurs consecutively (consecutive days).

C. Filter Criteria:

- Display only products that have experienced a consistent rising trend for at least 12 consecutive days.

D. Growth Calculation (Growth %):

- Calculate the growth percentage using the End Point vs Start Point Method for that trend session, with the formula:

$$\text{Growth \%} = \left(\frac{\text{Nilai MA Akhir Tren}}{\text{Nilai MA Awal Tren}} - 1 \right) \times 100$$

An example of the first row of output data from Rising Star is as follows (note: the number formatting follows the local settings of the problem creator's laptop):

	A	B	C	D	
1	Kode Produk	Nama Produk	Growth %	Total Penjualan	
2	MGR1L	Minyak Goreng Refill 1L	712,07	44.940.000	

Conditions for Generating Potential Packaging

The Potential Packaging sheet contains combinations of products frequently purchased together by customers, to be used for product bundling strategies, package promos, cross-selling, and so on.

For this hackathon, you are required to use the Apriori algorithm from the `mlxtend` package.

Here are several filtering conditions to obtain frequent itemsets matching Sophia's findings:

- Minimum support is 0.01, or 1% of total transactions to be processed.
- Form association rules with metric = 'lift', and min_threshold = 1.
- Not all rules are displayed. Only rules meeting the following requirements may be included in the final result:
 - At least one product in the rule must be a Rising Star product, either in the antecedents (origin product) or consequents (destination product).

- The "lift" value must be at least 2.
- The final result must be sorted from highest to lowest based on the following hierarchy:
 1. Lift
 1. Support
 2. Confidence

Below are some examples of the first row of output data resulting from the above filtering conditions.

	A	B	C	D	E	F
	Jika Membeli	Maka Membeli	Jumlah Invoice	Support	Confidence	Lift
2	Beras Premium 5kg	Minyak Goreng Refill 1L	100	0,01	0,26	2,3
3	Minyak Goreng Refill 1L	Beras Premium 5kg	100	0,01	0,1	2,3
4	Sabun Cuci Cair 1.5L, Kaos Kaki (3 Pasang)	Minyak Goreng Refill 1L	185	0,02	0,25	2,26
5	Minyak Goreng Refill 1L	Sabun Cuci Cair 1.5L, Kaos Kaki (3 Pasang)	185	0,02	0,18	2,26

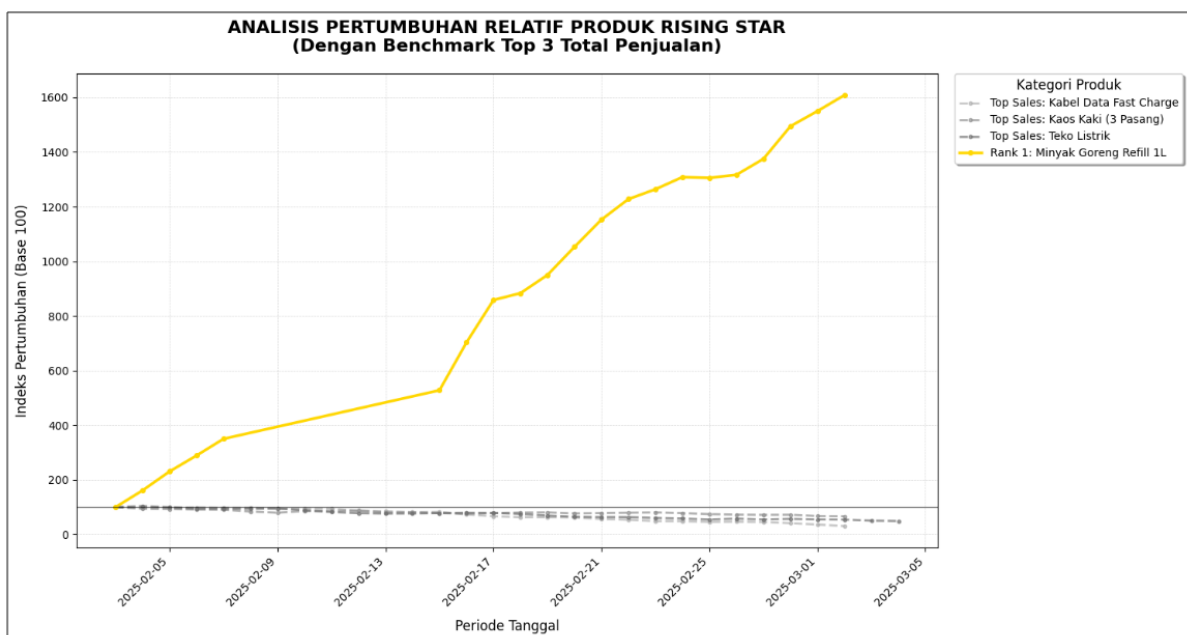
Rising Star Visualization Specifications (Index and Actual)

There are two types of visualizations for the rising star data.

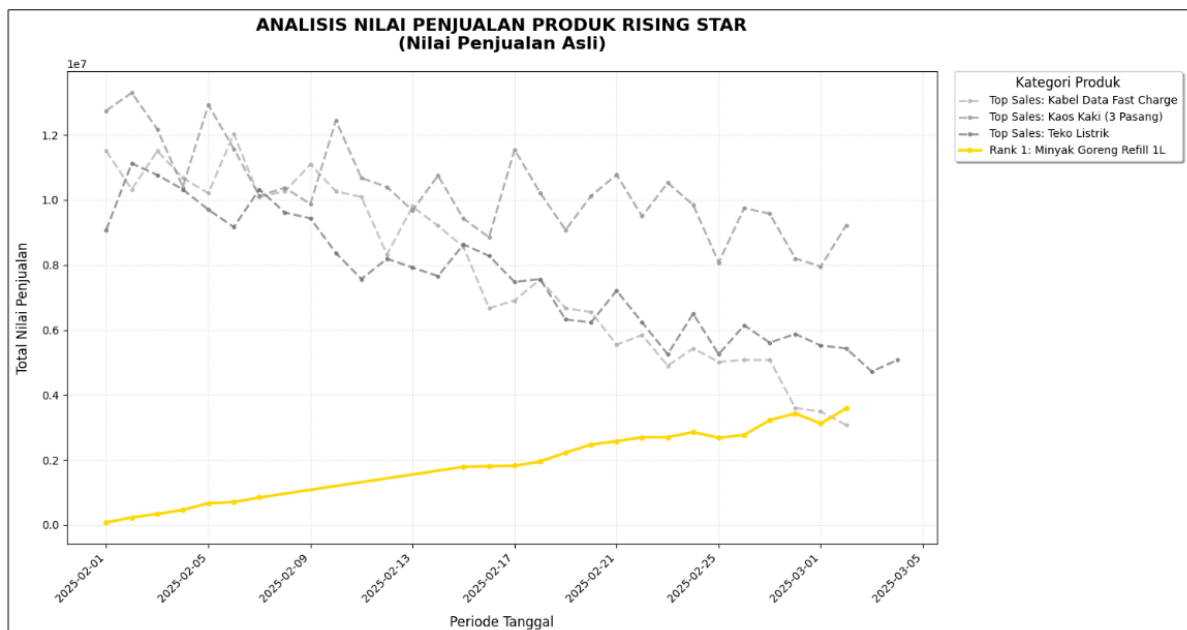
Relative Growth Visualization (Index / Normalized): Create a line chart for all rising star products as per the criteria above, compared with the top 3 products, with the following requirements:

Base 100 Normalization: Transform the MA value of each product so that all lines start at point 100 at the beginning of the observation period. This is intended so that the growth speed comparison between products appears visually fair.

1. **Y-axis:** Growth Index (Base 100).
2. **X-axis:** Date.
3. **Legend:** Display the Product Name (all products).



Actual Value Growth Visualization: You also need to produce a chart showing the actual values in addition to the growth index. Below is an example for the top three product sales together with the top 1 rising star product.



To help you, a file named `snippet_code_matplotlib.py` has been provided, which is a code snippet for you to analyze, used to generate that chart. However, note that this code cannot run standalone since it is only a code fragment.

The chart you produce must match exactly, or very closely, in terms of color, dimensions, and style used.

To give you an idea, the two incomplete chart files are also included with this email.

Python Version and Allowed Libraries

1. Python version 3.10 – 3.14
2. matplotlib version 3.10.7
3. pandas version 2.3.1
4. mlxtend version 0.23.4
5. openpyxl version 3.1.5